COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures & Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	09/24/2026	**DATE SUBMITTED:**	09/24/2026
NAME:	Acelajado, Vince Aguila, Daniel Alvarez, Joshua Caleb Baltazar, Kenji Cabangon, Mike Justin	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 9.1 Tree ADT

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.

Task 1: Create code in C++ that will create a tree as shown in the figure above. Use linked lists as the internal representation of this tree. Indicate your code screenshot and comments in table 9-1

CODE	SCREENSHOTS AND COMMENTS

GeneralTree.h

```h
//Alvarez,Joshua Caleb P.
#ifndef GENERAL_TREE_H
#define GENERAL_TREE_H

#include <iostream>

template <typename T>
class GeneralTree {
private:
    T key;

    GeneralTree<T>* child;
    GeneralTree<T>* sibling;

public:
    GeneralTree(T value);

    void insertChild(GeneralTree<T>* newChild);

    void preOrder();
};

template <typename T>
GeneralTree<T>::GeneralTree(T value) {
    key = value;
    child = nullptr;
    sibling = nullptr;
}
```

```cpp
template <typename T>
void GeneralTree<T>::insertChild(GeneralTree<T>* newChild) {

    if (child == nullptr) {
        child = newChild;
    }
    else {
        GeneralTree<T>* temp = child;

        while (temp->sibling != nullptr) {
            temp = temp->sibling;
        }

        temp->sibling = newChild;
    }
}

template <typename T>
void GeneralTree<T>::preOrder() {

    std::cout << key << " ";

    if (child != nullptr) {
        child->preOrder();
    }

    if (sibling != nullptr) {
        sibling->preOrder();
    }
}

#endif
```

This code shows how the structure of the general tree can be created using the linked list method. It consists of a node which holds the value, the first child pointer, and the sibling pointer. Insertion of nodes into the tree is done using the insertChild() method. The result is creation of the tree which has multiple children for each node.

Main.cpp

```cpp
#include <iostream>
#include "GeneralTree.h"

int main() {

    GeneralTree<char> A('A');
    GeneralTree<char> B('B');
    GeneralTree<char> C('C');
    GeneralTree<char> D('D');
    GeneralTree<char> E('E');
    GeneralTree<char> F('F');
    GeneralTree<char> G('G');

    GeneralTree<char> H('H');
    GeneralTree<char> I('I');
    GeneralTree<char> J('J');

    GeneralTree<char> K('K');
    GeneralTree<char> L('L');
    GeneralTree<char> M('M');
    GeneralTree<char> N('N');

    GeneralTree<char> P('P');
    GeneralTree<char> Q('Q');

```

```cpp
    // A children
    A.insertChild(&B);
    A.insertChild(&C);
    A.insertChild(&D);
    A.insertChild(&E);
    A.insertChild(&F);
    A.insertChild(&G);

```

```cpp
    // D child
    D.insertChild(&H);

    // E children
    E.insertChild(&I);
    E.insertChild(&J);

    // J children
    J.insertChild(&P);
    J.insertChild(&Q);
```

<table>
<tr><td></td><td>

```
51      // G child
52      G.insertChild(&N);
53
54      std::cout << "PreOrder: ";
55      A.preOrder();
56
57      return 0;
58  }
```

This code builds the nodes for the general tree and links them based on the tree diagram provided. The nodes are assigned letters in accordance with the tree. The insertChild method is used to make the links between parents and children. Once all the nodes have been linked, the algorithm begins its traversal from node A.
</td></tr>
<tr><td>OUTPUT</td><td>

This is a program that implements the traversal of a general tree using PreOrder, InOrder, and PostOrder algorithms. The <code>child</code> pointer is treated as the "left" branch while the <code>sibling</code> pointer is treated as the "right" branch, allowing standard binary traversal logic to be applied to a tree with any number of children.

```
[Running] cd "c:\Users\RYZEN 5\OneDrive\DSA\HOA\
5\OneDrive\DSA\HOA\"CPE010_HOA9p1_Group1_sc1_mai
PreOrder: A B C D H E I J P Q F K L M G N
InOrder: B C H D I P Q J E K L M F N G A
PostOrder: H Q P J I M L K N G F E D C B A
```
</td></tr>
</table>

Table 9.1. General Tree Code Implementation and Comments

TABLE

NODE	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1

E	**2**	**1**
F	**1**	**1**
G	**1**	**1**
H	**0**	**2**
I	**0**	**2**
J	**1**	**2**
K	**0**	**2**
L	**0**	**2**
M	**0**	**2**
N	**0**	**2**
P	**0**	**3**
Q	**0**	**3**

Table 9-2

ILO B

TABLE

Pre-Order	A, B, C, D, H, E, I, J, P, Q, F, K, L, M, G, N
Post-Order	B, C, H, D, I, P, Q, J, E, K, L, M, F, N, G, A
In-Order	B, A, C, H, D, I, E, P, J, Q, K, F, L, M, N, G

Table 9-3

TABLE

Pre-Order	20, 10, 15, 30, 70, 50, 40, 60, 80

Post-Order	15, 10, 40, 60, 50, 80, 70, 30, 20
In-Order	10, 15, 20, 30, 40, 50, 60, 70, 80

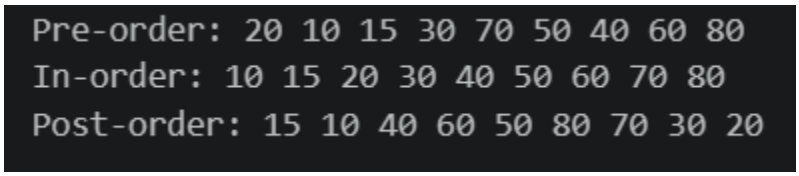


Table 9-4

OBSERVATION

The outputs of the functions are the same as the results obtained by manually traversing the tree. The pre-order follows Root-Left-Right, the in-order follows Left-Root-Right, and the post-order follows Left-Right-Root. This shows that the three traversal functions were implemented correctly.

TABLE

Main.cpp

```cpp
#include <iostream>
#include "BST.h"

int main() {

    BinarySearchTree<int> tree(20);

    tree.insert(30);
    tree.insert(70);
    tree.insert(50);
    tree.insert(40);
    tree.insert(60);
    tree.insert(80);
    tree.insert(10);
    tree.insert(15);

    std::cout << "Pre-order search: ";
    tree.findData(1, 60);

    std::cout << "In-order search: ";
    tree.findData(2, 60);

    std::cout << "Post-order search: ";
    tree.findData(3, 60);

    return 0;
```

Bst.h

```cpp
#ifndef BST_H
#define BST_H

#include <iostream>

template <typename T>
class BinarySearchTree {

private:
    struct Node {
        T data;
        Node* left;
        Node* right;

        Node(T value) {
            data = value;
            left = nullptr;
            right = nullptr;
        }
    };
```

```
21
22      Node* root;
23
24      void insert(Node*& node, T value) {
25          if (node == nullptr) {
26              node = new Node(value);
27          }
28          else if (value < node->data) {
29              insert(node->left, value);
30          }
31          else {
32              insert(node->right, value);
33          }
34      }
35
36      bool findPreOrder(Node* node, T key) {
37          if (node == nullptr)
38              return false;
39
40          if (node->data == key)
41              return true;
42
43          if (findPreOrder(node->left, key))
44              return true;
46          return findPreOrder(node->right, key);
47      }
48
49      bool findInOrder(Node* node, T key) {
50          if (node == nullptr)
51              return false;
52
53          if (findInOrder(node->left, key))
54              return true;
55
56          if (node->data == key)
57              return true;
58
59          return findInOrder(node->right, key);
60      }
61
62      bool findPostOrder(Node* node, T key) {
63          if (node == nullptr)
64              return false;
65
66          if (findPostOrder(node->left, key))
67              return true;
68
69          if (findPostOrder(node->right, key))
70              return true;
```

```
70                  return true;
71
72              if (node->data == key)
73                  return true;
74
75              return false;
76          }
77
78      public:
79          BinarySearchTree(T value) {
80              root = new Node(value);
81          }
82
83          void insert(T value) {
84              insert(root, value);
85          }
86
87          void findData(int choice, T key) {
88
89              bool found = false;
90
91              if (choice == 1) {
92                  found = findPreOrder(root, key);
93              }
94              else if (choice == 2) {
95                  found = findInOrder(root, key);
96              }
97              else if (choice == 3) {
98                  found = findPostOrder(root, key);
```

```
98                  found = findPostOrder(root, key);
99              }
100
101             if (found) {
102                 std::cout << key << " was found." << std::endl;
103             }
104         }
105     };
106
107     #endif
```

```
Pre-order search: 60 was found.
In-order search: 60 was found.
Post-order search: 60 was found.
```

Table 9-5

OBSERVATION
The findData function successfully searches for the given key based on the selected traversal method. When the key exists in the tree, it displays that the key was found. When the key does not exist, the function produces no output as required.

TABLE

Bst.h

```
87      void findData(int choice, T key) {
88
89          bool found = false;
90
91          if (choice == 1) {
92              found = findPreOrder(root, key);
93          }
94          else if (choice == 2) {
95              found = findInOrder(root, key);
96          }
97          else if (choice == 3) {
98              found = findPostOrder(root, key);
99          }
100
101         if (found) {
102             std::cout << key << " was found." << std::endl;
103         }
104     }
105 };
106
107 #endif
```

Main.cpp

```cpp
int main() {

    BinarySearchTree<char> tree('A');

    tree.insert('B');
    tree.insert('C');
    tree.insert('D');
    tree.insert('E');
    tree.insert('F');
    tree.insert('G');
    tree.insert('H');

    // Add O as a new leaf node
    tree.insert('O');

    std::cout << "Pre-order search: ";
    tree.findData(1, 'O');

    std::cout << "In-order search: ";
    tree.findData(2, 'O');

    std::cout << "Post-order search: ";
    tree.findData(3, 'O');

    return 0;
```

Output

```
Pre-order search: O was found.
In-order search: O was found.
Post-order search: O was found.
```

OBSERVATION
After adding O as a child of node G, the findData() function is still able to find the new node. The output confirms that O exists in the tree. The traversal method changes the order in which the nodes are visited, but all three methods can still find O.

2. ANSWER TO SUPPLEMENTARY ACTIVITY

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

CODE

```cpp
#include <iostream>
#include "BST.h"

int main(){
    BinarySearchTree<int> tree(2);

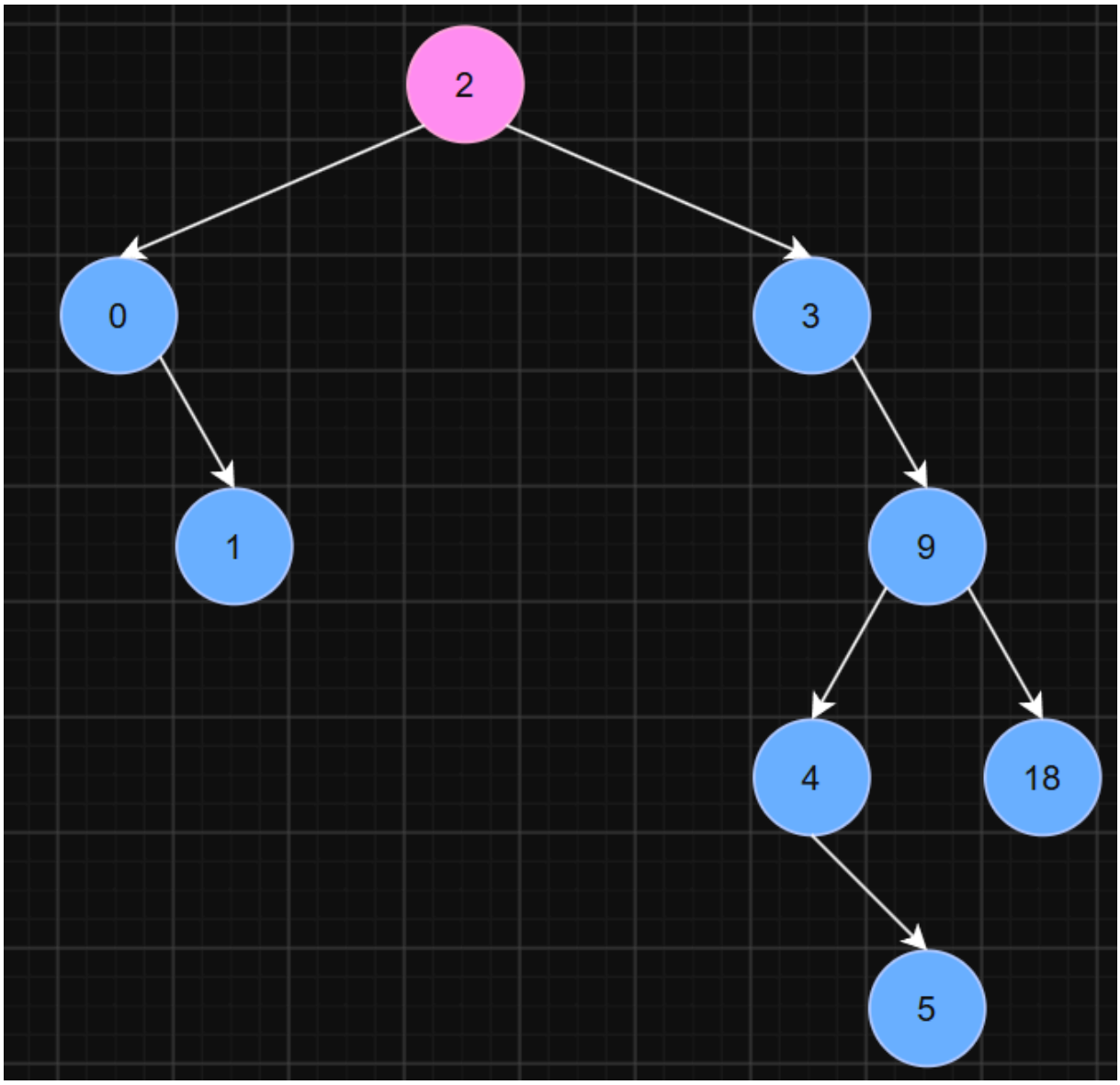
    std::cout<<"Pre Order: ";
    tree.insert(2);
    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);

    tree.preOrder();
    std::cout<<"\nIn Order: ";
    tree.inOrder();
    std::cout<<"\nPost Order: ";
    tree.postOrder();

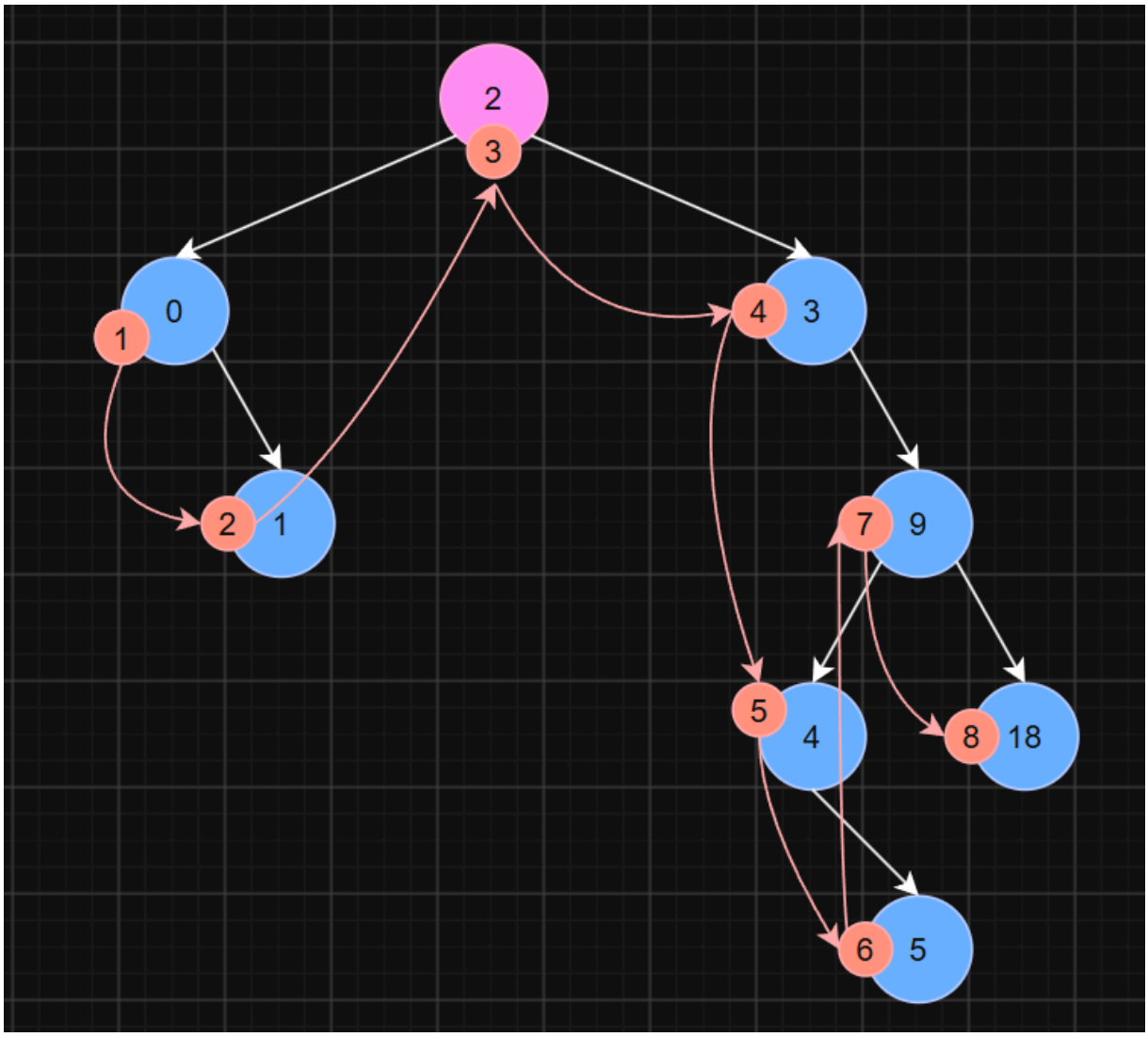
    return 0;
}
```

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.

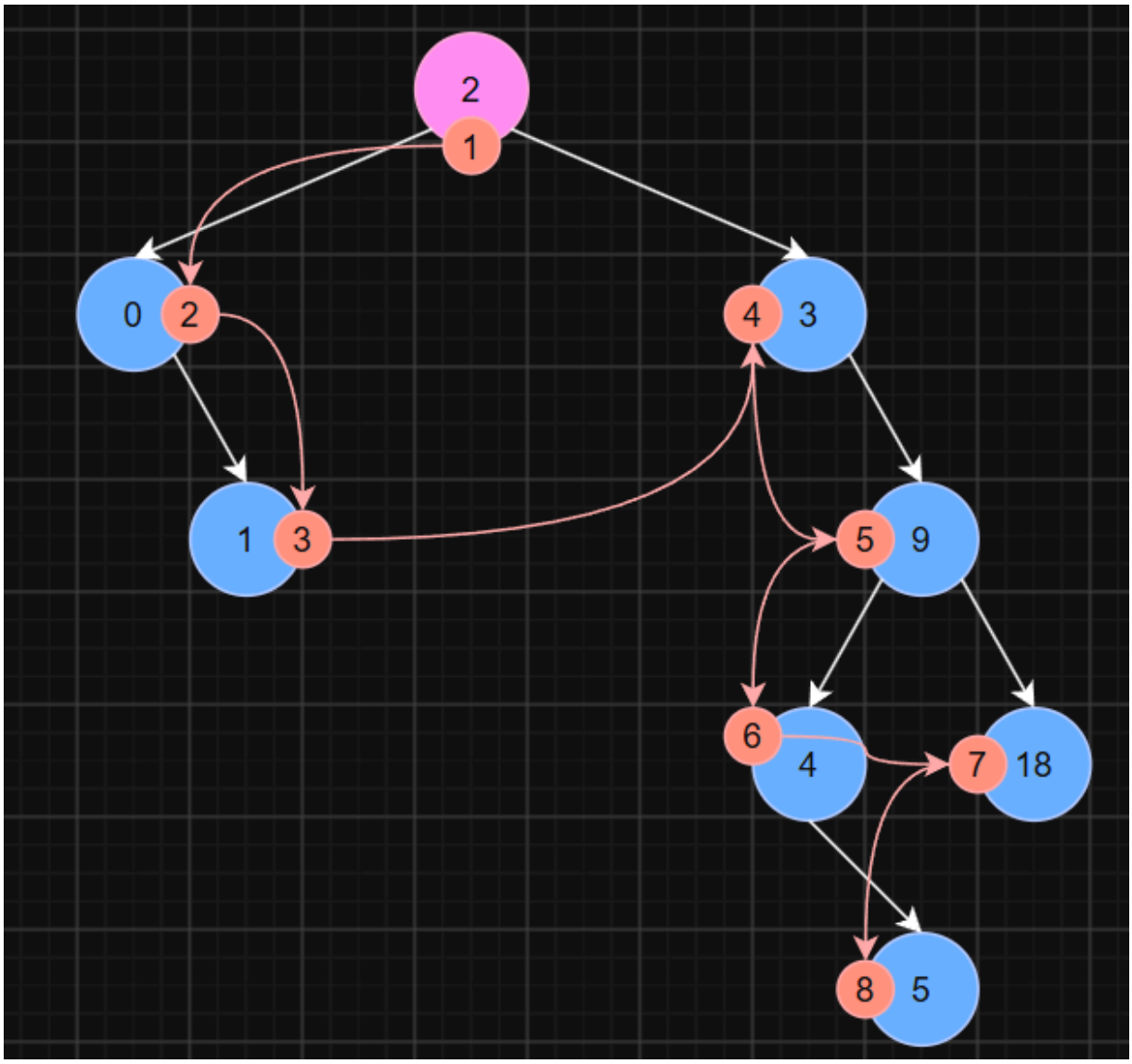
TREE DIAGRAM



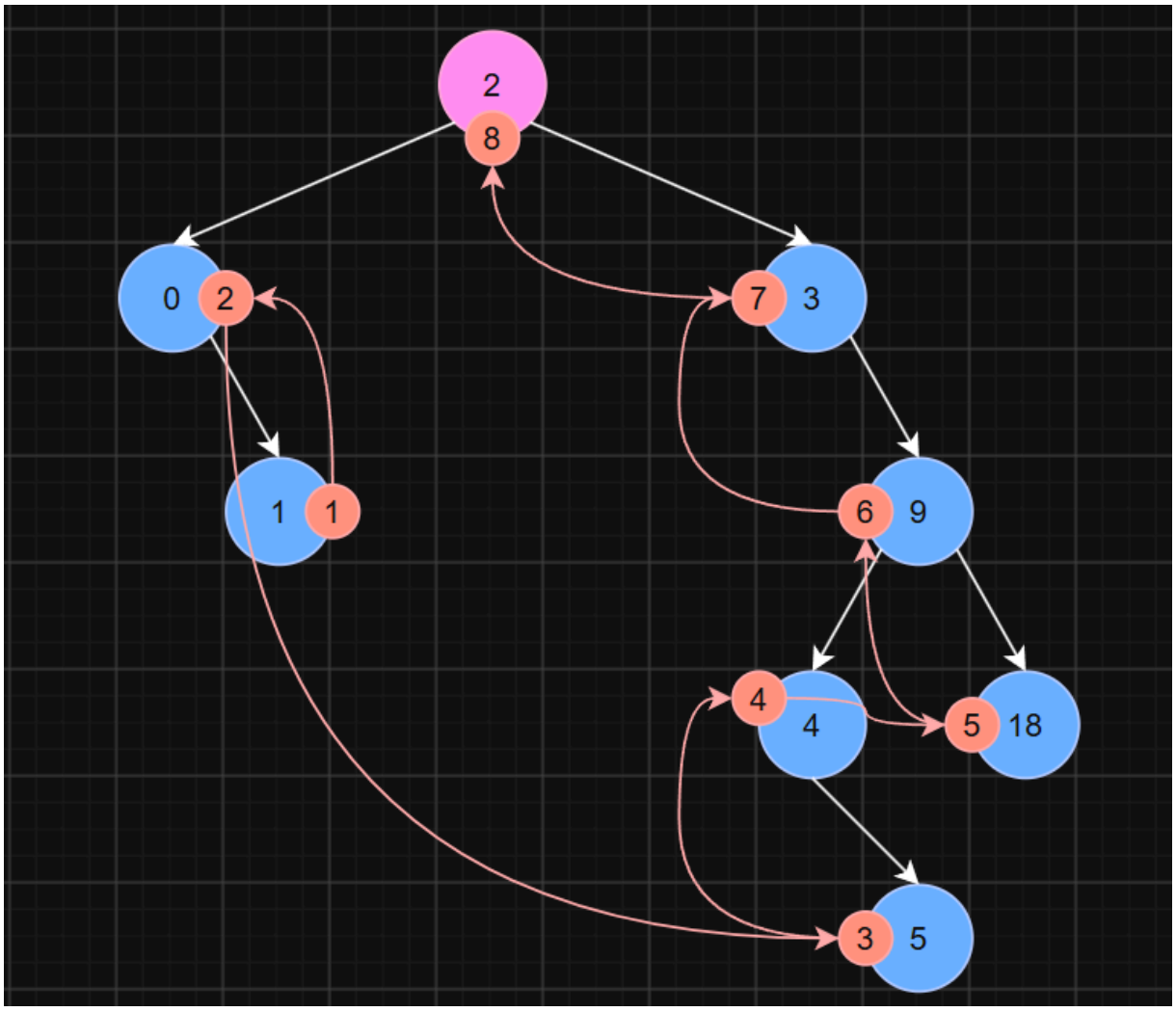
IN ORDER



PRE ORDER



POST ORDER



Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

FUNCTION

```
template <typename T>
void BinarySearchTree<T>::inOrder(){
    if(left != nullptr){
        left->inOrder();
    }
    std::cout<<key<< " ";
    if(right != nullptr){
        right->inOrder();
    }
}
```

OUTPUT

```
In Order: 0 1 2 3 4 5 9 18
```

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

Pre-order Traversal

FUNCTION

```
template <typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout<<key<< " ";
    if(left != nullptr){
        left->preOrder();
    }
    if(right != nullptr){
        right->preOrder();
    }
}
```

OUTPUT

```
Pre Order: 2 0 1 3 9 4 5 18
```

Post-order Traversal

FUNCTION

```
template <typename T>
void BinarySearchTree<T>::postOrder(){
    if(left != nullptr){
        left->postOrder();
    }
    if(right != nullptr){
        right->postOrder();
    }
    std::cout<<key<< " ";
}
```

OUTPUT

```
Post Order: 1 0 5 4 18 9 3 2
```

3. CONCLUSION

This particular task involved the creation and navigation of both a general tree, wherein a node could have an arbitrary number of child nodes stored through a header file (GeneralTree.h) different from main.cpp, as well as a binary search tree, which had at most two child nodes ordered based on their values; and pre-order, in-order and post-order tree traversal using recursion in each case proved that in-order traversal has a natural meaning only for binary trees, which requires a general formula in other cases, and also proved why in-order traversal is the way to go with a BST.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.